Ledbetter Data Pump (LDP)

Phase 1 Operational Details

Final clarifying note for Phase 1

Ledbetter Data Pump (LDP)

Phase 1 Operational Details

Version 1.0

Date: November 2026

Status: Final clarifying note for Phase 1

This short note locks down five operational details that were previously implied.

It is intended to be used together with the Architecture White Paper, Failure Modes Addendum, and Phase 1 Implementation Contract.

1. Range-Partition Planning Algorithm

When `partitioning.type: range`:

1. Run a single planning query:

`SELECT MIN(column), MAX(column) FROM () AS src`

(The engine wraps the user query as a subquery.)

2. If the result is `NULL, NULL` (empty table or all-null column):

- Produce **zero partitions**.

- Job succeeds immediately with 0 rows.

3. Otherwise let `min_val` and `max_val` be the returned integers.

Compute chunk size:

`chunk_size = ((max_val - min_val) / chunks) + 1`

(integer division, at least 1).

4. Generate partitions:

```

for i in 0..chunks:

start = min_val + i * chunk_size
```

end = min(start + chunk_size, max_val + 1)

if start < end:

emit Partition { id: i, start: Some(start), end: Some(end) }

```

5. The source connector must execute:

`SELECT ... FROM () AS src WHERE column >= $1 AND column < $2`

6. Non-integer partition columns are rejected at planning time in Phase 1.

# 2. Concrete Log Event Schema

All logs are single-line JSON to stdout. Required fields are always present.

## chunk_started

```
{
  "timestamp": "2026-11-10T18:03:00Z",
  "level": "INFO",
  "event": "chunk_started",
  "pipeline": "order_sync",
  "chunk_id": 3,
  "partition_start": 1000000,
  "partition_end": 1250000
}
```

## chunk_completed

```
{
  "timestamp": "2026-11-10T18:03:01Z",
  "level": "INFO",
  "event": "chunk_completed",
  "pipeline": "order_sync",
  "chunk_id": 3,
  "rows": 24891,
  "duration_ms": 842,
  "rows_per_sec": 29561
}
```

job_summary (always emitted, success or failure)

```
{
  "timestamp": "2026-11-10T18:03:45Z",
  "level": "INFO",
  "event": "job_summary",
  "pipeline": "order_sync",
  "status": "success",
  "chunks_total": 16,
  "chunks_completed": 16,
  "rows_read": 1_000_000,
  "rows_written": 987_654,
  "duration_ms": 45210,
  "error": null
}
```

error
```

```
{
  "timestamp": "2026-11-10T18:03:12Z",
  "level": "ERROR",
  "event": "error",
  "pipeline": "order_sync",
  "chunk_id": 7,
  "message": "source error: connection reset"
}
```

3. CLI Contract

```
ldp run <pipeline.yaml> [OPTIONS]
```

Options (Phase 1)

• `--threads ` — overrides `settings.max_threads`

• `--state-file ` — overrides state file location

• `--log-level ` — `error|warn|info|debug` (default `info`)

Exit Codes

No other subcommands are required in Phase 1.

4. Minimal Guaranteed-Working Example

Postgres DDL

```
CREATE TABLE customers (
    id          BIGINT PRIMARY KEY,
    name        TEXT,
    status      TEXT,
    amount      DOUBLE PRECISION,
    created_at  TIMESTAMP
);

-- Optional seed data for testing
INSERT INTO customers (id, name, status, amount, created_at)
SELECT generate_series(1, 100000),
       'customer_' || generate_series(1, 100000),
       CASE WHEN random() > 0.1 THEN 'ACTIVE' ELSE 'CANCELLED' END,
       random() * 1000,
       NOW() - (random() * INTERVAL '365 days');
```

pipeline.yaml

```
version: "1.0"
pipeline:
  name: "customer_export"
  settings:
    max_threads: 4
    state_file: ".ldp/customer_export.json"
  resources:
    connections:
      - name: "pg"
        type: "postgres"
        url: "env://PG_URL"
  dataflow:
    sources:
      - id: "src"
        connection: "pg"
```

```
    query: "SELECT id, name, status, amount, created_at FROM customers"
    partitioning:
      type: "range"
      column: "id"
      chunks: 4
transformations:
  - id: "active_only"
    input: "src"
    type: "filter"
    condition: "status = 'ACTIVE' AND amount > 0"
targets:
  - id: "out"
    input: "active_only"
    type: "parquet"
    path: "./output/customers/"
    compression: "zstd"
```

This example must pass the Phase 1 Definition of Done (including restart).

5. Connection String & Secret Handling

Rules (Phase 1)

• Supported URL forms:

- `env://VARIABLE_NAME`

- `postgres://user:pass@host:port/db` (plain)

• The engine **never** logs a full connection URL.

• When a URL must appear in a log or error message, redact the password:

`postgres://user:***@host:port/db`

• Environment variables are resolved at startup. Missing variables produce a configuration error (exit code 1).

• No other secret mechanisms (files, vaults, etc.) are supported in Phase 1.

End of Phase 1 Operational Details

These five points are now fully specified. No further clarification is required for Phase 1 implementation.